cmd\discv4demo\main.go

```go
package main

import (
    "bytes"
    "crypto/ecdsa"
    "encoding/hex"
    "errors"
    "flag"
    "fmt"
    "log"
    "net"
    "net/netip"
    "sort"
    "strings"
    "sync"
    "time"

    "github.com/ethereum/go-ethereum/crypto"
    "github.com/ethereum/go-ethereum/p2p/discover/v4wire"
    "github.com/ethereum/go-ethereum/p2p/enode"
    "github.com/ethereum/go-ethereum/p2p/enr"

    "github.com/ethereum/go-ethereum/cmd/internal/kademlia"
)

// 이 데모는 로컬에서 여러 discv4 에이전트를 띄워 실제 UDP 패킷으로 핑/파인드노드/네이버스 등을 주고받으며
// Kademlia 라우팅 테이블이 어떻게 구축되고 iterative lookup이 동작하는지 관찰할 수 있게 한다.

// main은 여러 에이전트를 가동하고 서로 핑을 교환한 뒤 Kademlia lookup 시나리오를 실행한다.
func main() {
    agentCount := flag.Int("agents", 4, "number of local discovery agents to start")
    basePort := flag.Int("baseport", 30400, "UDP port to start allocating from")
    alpha := flag.Int("alpha", 2, "parallelism for iterative lookups")
    flag.Parse()

    if *agentCount < 2 {
        log.Fatalf("need at least two agents for the demo")
    }

    agents := make([]*agent, 0, *agentCount)
    var wg sync.WaitGroup
    for i := 0; i < *agentCount; i++ {
        port := *basePort + i
        name := fmt.Sprintf("A%d", i+1)
        ag, err := newAgent(name, port)
        if err != nil {
            log.Fatalf("cannot create agent %s: %v", name, err)
        }
        agents = append(agents, ag)
    }
```

```go

    for _, ag := range agents {
        wg.Add(1)
        go ag.readLoop(&wg)
    }

    // 모든 에이전트가 서로 핑을 주고받도록 돌려 NAT 바깥에서 하듯 본딩 과정을 흉내 낸다.
    for i := range agents {
        for j := range agents {
            if i == j {
                continue
            }
            if err := agents[i].sendPing(agents[j].udpAddr()); err != nil {
                log.Fatalf("%s failed to ping %s: %v", agents[i].name, agents[j].name, err)
            }
            time.Sleep(30 * time.Millisecond)
        }
    }

    time.Sleep(200 * time.Millisecond)

    // 라우팅 테이블은 enode.ID를 키로 사용하므로 에이전트도 ID → 인스턴스 매핑을 준비한다.
    idToAgent := make(map[enode.ID]*agent)
    for _, ag := range agents {
        idToAgent[ag.id] = ag
    }

    source := agents[0]
    target := agents[len(agents)-1]

    fmt.Printf("Running iterative lookup from %s to %s (%s)\n", source.name, target.name, shortID(target.id))

    report, err := iterativeLookup(source, target, idToAgent, *alpha)
    if err != nil {
        log.Fatalf("lookup failed: %v", err)
    }

    // 단계별 탐색 결과와 최종 테이블 상태를 출력해 학습자가 흐름을 따라가도록 한다.
    fmt.Println("Lookup steps:")
    for i, step := range report.Steps {
        fmt.Printf("  Step %d: queried %s, discovered %s\n", i+1, step.QueriedDescription(), step.DiscoveredDescription())
    }
    fmt.Println()

    fmt.Println("Final routing tables:")
    for _, ag := range agents {
        fmt.Printf("  %s: %d nodes across %d buckets\n", ag.name, ag.table.Len(), len(ag.table.Buckets()))
    }

    for _, ag := range agents {
```

```go
        ag.close()
    }
    wg.Wait()
}

// agent는 discv4 에이전트 한 개를 추상화하며, 로컬 키/소켓과 라우팅 테이블, 대기 중인 요청을 관리한다.
type agent struct {
    name     string
    priv     *ecdsa.PrivateKey
    pubkey   v4wire.Pubkey
    id       enode.ID
    conn     *net.UDPConn
    endpoint v4wire.Endpoint
    table    *kademlia.Table
    record   *enr.Record

    mu          sync.Mutex
    pendingFind map[enode.ID]*findRequest
}

// findRequest는 보내둔 findnode 요청에 대응하는 neighbors 응답을 기다릴 때 사용하는 구조체다.
type findRequest struct {
    target kademlia.ID
    ch     chan []kademlia.Node
}

// newAgent는 새로운 키를 만들고 UDP 리스너를 띄운 뒤, 라우팅 테이블과 ENR을 초기화한다.
func newAgent(name string, port int) (*agent, error) {
    priv, err := crypto.GenerateKey()
    if err != nil {
        return nil, fmt.Errorf("generate key: %w", err)
    }
    addr := &net.UDPAddr{IP: net.ParseIP("127.0.0.1"), Port: port}
    conn, err := net.ListenUDP("udp", addr)
    if err != nil {
        return nil, fmt.Errorf("listen UDP: %w", err)
    }
    local := conn.LocalAddr().(*net.UDPAddr)
    endpoint := endpointFromUDPAddr(local)
    pubkey := v4wire.EncodePubkey(&priv.PublicKey)
    id := pubkey.ID()

    record, err := buildRecord(priv, endpoint)
    if err != nil {
        _ = conn.Close()
        return nil, err
    }

    ag := &agent{
        name:        name,
        priv:        priv,
        pubkey:      pubkey,
```

```go
        id:          id,
        conn:        conn,
        endpoint:    endpoint,
        table:       kademlia.NewTable(id, kademliaBucketSize),
        record:      record,
        pendingFind: make(map[enode.ID]*findRequest),
    }
    log.Printf("[%s] listening on %s", ag.name, local)
    return ag, nil
}

const (
    readBufSize        = 2048
    responseTimeout    = 2 * time.Second
    kademliaBucketSize = 16
)

func (a *agent) close() {
    _ = a.conn.Close()
}

func (a *agent) udpAddr() *net.UDPAddr {
    return a.conn.LocalAddr().(*net.UDPAddr)
}

// readLoop는 UDP 소켓에서 들어오는 패킷을 계속 읽어 decode 후 패킷 핸들러로 넘긴다.
func (a *agent) readLoop(wg *sync.WaitGroup) {
    defer wg.Done()
    buf := make([]byte, readBufSize)
    for {
        n, addr, err := a.conn.ReadFromUDP(buf)
        if err != nil {
            if errors.Is(err, net.ErrClosed) {
                log.Printf("[%s] socket closed", a.name)
                return
            }
            log.Printf("[%s] read error: %v", a.name, err)
            continue
        }
        data := make([]byte, n)
        copy(data, buf[:n])
        msg, fromKey, hash, err := v4wire.Decode(data)
        if err != nil {
            log.Printf("[%s] failed to decode packet from %s: %v", a.name, addr, err)
            continue
        }
        remoteID := fromKey.ID()
        a.handlePacket(addr, msg, fromKey, remoteID, hash)
    }
}

// handlePacket은 패킷 종류에 따라 라우팅 테이블 갱신, 응답 전송, 대기 중인 요청 해제를 수행한다.
```

```go
func (a *agent) handlePacket(addr *net.UDPAddr, msg v4wire.Packet, fromKey v4wire.Pubkey, remoteID enode.ID, hash []byte) {
    switch packet := msg.(type) {
    case *v4wire.Ping:
        log.Printf("[%s] <- PING from %s (%s)", a.name, addr, shortID(remoteID))
        if !v4wire.Expired(packet.Expiration) {
            node := nodeFromPing(fromKey, packet.From)
            node.Label = addr.String()
            a.table.AddNode(node)
            if err := a.sendPong(addr, hash); err != nil {
                log.Printf("[%s] failed to send PONG: %v", a.name, err)
            }
        }
    case *v4wire.Pong:
        log.Printf("[%s] <- PONG from %s (%s)", a.name, addr, shortID(remoteID))
    case *v4wire.Findnode:
        target := packet.Target.ID()
        log.Printf("[%s] <- FINDNODE from %s target=%s", a.name, addr, shortID(target))
        neighbors := a.table.FindnodeByID(target, kademliaBucketSize)
        if len(neighbors) == 0 {
            log.Printf("[%s] no neighbors to send", a.name)
            return
        }
        a.sendNeighbors(addr, neighbors)
    case *v4wire.Neighbors:
        nodes := make([]kademlia.Node, 0, len(packet.Nodes))
        for _, rn := range packet.Nodes {
            node := kademlia.FromV4Node(rn)
            node.Label = fmt.Sprintf("%s:%d", net.IP(rn.IP), rn.UDP)
            a.table.AddNode(node)
            nodes = append(nodes, node)
        }
        a.mu.Lock()
        if req := a.pendingFind[remoteID]; req != nil {
            delete(a.pendingFind, remoteID)
            req.ch <- nodes
        }
        a.mu.Unlock()
        log.Printf("[%s] <- NEIGHBORS from %s (%d nodes)", a.name, addr, len(nodes))
    case *v4wire.ENRRequest:
        log.Printf("[%s] <- ENRREQUEST from %s", a.name, addr)
        a.sendENRResponse(addr, hash)
    case *v4wire.ENRResponse:
        log.Printf("[%s] <- ENRRESPONSE from %s", a.name, addr)
    default:
        log.Printf("[%s] <- %s from %s", a.name, msg.Name(), addr)
    }
}

// sendPing은 Kademlia 본딩 과정과 동일하게 ping 패킷을 보내고 로그를 남긴다.
func (a *agent) sendPing(remote *net.UDPAddr) error {
    ping := &v4wire.Ping{
        Version:    4,
        From:       a.endpoint,
```

```go
        To:         endpointFromUDPAddr(remote),
        Expiration: futureExpiration(),
    }
    _, err := a.sendPacket(ping, remote)
    if err == nil {
        log.Printf("[%s] -> PING %s", a.name, remote)
    }
    return err
}

// sendPong은 받은 ping에 대한 응답으로 pong을 보낸다.
func (a *agent) sendPong(remote *net.UDPAddr, replyTok []byte) error {
    pong := &v4wire.Pong{
        To:         endpointFromUDPAddr(remote),
        ReplyTok:   replyTok,
        Expiration: futureExpiration(),
    }
    _, err := a.sendPacket(pong, remote)
    if err == nil {
        log.Printf("[%s] -> PONG %s", a.name, remote)
    }
    return err
}

// sendNeighbors는 테이블에서 가까운 노드를 골라 neighbors 패킷으로 전달한다.
func (a *agent) sendNeighbors(remote *net.UDPAddr, nodes []kademlia.Node) {
    payload := &v4wire.Neighbors{
        Nodes:      make([]v4wire.Node, 0, len(nodes)),
        Expiration: futureExpiration(),
    }
    for _, node := range nodes {
        if node.Pubkey == (v4wire.Pubkey{}) {
            continue
        }
        payload.Nodes = append(payload.Nodes, kademlia.ToV4Node(node))
    }
    if _, err := a.sendPacket(payload, remote); err != nil {
        log.Printf("[%s] failed to send NEIGHBORS: %v", a.name, err)
    } else {
        log.Printf("[%s] -> NEIGHBORS %s (%d nodes)", a.name, remote, len(payload.Nodes))
    }
}

// sendENRResponse는 상대가 요청한 ENR을 그대로 돌려준다.
func (a *agent) sendENRResponse(remote *net.UDPAddr, replyTok []byte) {
    resp := &v4wire.ENRResponse{ReplyTok: replyTok, Record: *a.record}
    if _, err := a.sendPacket(resp, remote); err != nil {
        log.Printf("[%s] failed to send ENRRESPONSE: %v", a.name, err)
    }
}

// sendFindnode는 상대에게 특정 타깃에 가까운 노드를 요청하고, 응답 채널을 준비한다.
func (a *agent) sendFindnode(remote *net.UDPAddr, remoteID enode.ID, target v4wire.Pubkey)
([]kademlia.Node, error) {
```

```go
    req := &v4wire.Findnode{Target: target, Expiration: futureExpiration()}
    rchan := make(chan []kademlia.Node, 1)

    a.mu.Lock()
    a.pendingFind[remoteID] = &findRequest{target: target.ID(), ch: rchan}
    a.mu.Unlock()

    _, err := a.sendPacket(req, remote)
    if err != nil {
        a.mu.Lock()
        delete(a.pendingFind, remoteID)
        a.mu.Unlock()
        return nil, err
    }
    log.Printf("[%s] -> FINDNODE %s (%s) target=%s", a.name, remote, shortID(remoteID), shortID(target.ID()))

    select {
    case nodes := <-rchan:
        return nodes, nil
    case <-time.After(responseTimeout):
        a.mu.Lock()
        delete(a.pendingFind, remoteID)
        a.mu.Unlock()
        return nil, fmt.Errorf("timeout waiting for neighbors")
    }
}

func (a *agent) sendPacket(msg v4wire.Packet, remote *net.UDPAddr) ([]byte, error) {
    packet, hash, err := v4wire.Encode(a.priv, msg)
    if err != nil {
        return nil, err
    }
    if _, err := a.conn.WriteToUDP(packet, remote); err != nil {
        return nil, err
    }
    return hash, nil
}

// buildRecord는 로컬 엔드포인트 정보를 담은 ENR을 만들어 서명한다.
func buildRecord(priv *ecdsa.PrivateKey, endpoint v4wire.Endpoint) (*enr.Record, error) {
    var record enr.Record
    if endpoint.IP != nil {
        record.Set(enr.IP(endpoint.IP))
    }
    if endpoint.UDP != 0 {
        record.Set(enr.UDP(endpoint.UDP))
    }
    if endpoint.TCP != 0 {
        record.Set(enr.TCP(endpoint.TCP))
    }
    if err := enode.SignV4(&record, priv); err != nil {
        return nil, err
    }
```

```go
    return &record, nil
}

// nodeFromPing은 받은 ping 패킷에서 상대 정보를 꺼내 라우팅 테이블 항목으로 변환한다.
func nodeFromPing(pub v4wire.Pubkey, ep v4wire.Endpoint) kademlia.Node {
    addr, _ := netip.AddrFromSlice(ep.IP)
    return kademlia.Node{
        ID:     pub.ID(),
        Pubkey: pub,
        IP:     addr,
        UDP:    ep.UDP,
        TCP:    ep.TCP,
    }
}

// endpointFromUDPAddr는 net.UDPAddr을 v4wire가 쓰는 endpoint 구조로 바꾼다.
func endpointFromUDPAddr(addr *net.UDPAddr) v4wire.Endpoint {
    ap := addrPort(addr)
    return v4wire.NewEndpoint(ap, uint16(addr.Port))
}

func addrPort(addr *net.UDPAddr) netip.AddrPort {
    if addr == nil {
        return netip.AddrPort{}
    }
    ip := addr.IP
    if ip == nil {
        ip = net.IPv4zero
    }
    if ip4 := ip.To4(); ip4 != nil {
        ip = ip4
    } else {
        ip = ip.To16()
    }
    if ip == nil {
        return netip.AddrPort{}
    }
    addrIP, ok := netip.AddrFromSlice(ip)
    if !ok {
        return netip.AddrPort{}
    }
    return netip.AddrPortFrom(addrIP, uint16(addr.Port))
}

// futureExpiration은 짧은 TTL을 가진 만료 시각을 만들어 패킷에 넣는다.
func futureExpiration() uint64 {
    return uint64(time.Now().Add(5 * time.Second).Unix())
}

// shortID는 로그 출력 시 ID 앞부분만 표시해 가독성을 높인다.
func shortID(id enode.ID) string {
    return hex.EncodeToString(id[:4])
}

```

```go
/* lookup structures */

// lookupReport는 iterative lookup 과정의 각 스텝을 모아둔 결과다.
type lookupReport struct {
    Steps []lookupStep
}

// lookupStep은 특정 라운드에서 질의한 에이전트와 신규로 발견한 노드 목록을 기록한다.
type lookupStep struct {
    Queried    []*agent
    Discovered []kademlia.Node
}

// lookupEntry는 shortlist에서 한 후보 노드와 해당 에이전트, 질의 여부를 묶는다.
type lookupEntry struct {
    node    kademlia.Node
    agent   *agent
    queried bool
}

func (ls lookupStep) QueriedDescription() string {
    if len(ls.Queried) == 0 {
        return "(none)"
    }
    names := make([]string, len(ls.Queried))
    for i, ag := range ls.Queried {
        names[i] = ag.name
    }
    return fmt.Sprintf("[%s]", strings.Join(names, ", "))
}

func (ls lookupStep) DiscoveredDescription() string {
    if len(ls.Discovered) == 0 {
        return "(none)"
    }
    labels := make([]string, len(ls.Discovered))
    for i, node := range ls.Discovered {
        if node.Label != "" {
            labels[i] = node.Label
        } else {
            labels[i] = shortID(node.ID)
        }
    }
    return fmt.Sprintf("[%s]", strings.Join(labels, ", "))
}

// iterativeLookup은 로컬 에이전트에서 시작해 α 병렬로 findnode를 보내며 타깃에 수렴한다.
func iterativeLookup(source, target *agent, registry map[enode.ID]*agent, alpha int)
(lookupReport, error) {
    targetID := target.id
    shortlist := make([]lookupEntry, 0)
    seen := make(map[enode.ID]bool)

    addNode := func(node kademlia.Node) {
```

```go
        if seen[node.ID] {
            return
        }
        ag := registry[node.ID]
        if ag == nil {
            return
        }
        seen[node.ID] = true
        shortlist = append(shortlist, lookupEntry{node: node, agent: ag})
    }

    initial := source.table.FindnodeByID(targetID, kademliaBucketSize)
    for _, node := range initial {
        addNode(node)
    }

    if len(shortlist) == 0 {
        addNode(kademlia.Node{ID: targetID, Pubkey: target.pubkey, IP: addrToNetip(target.udpAddr().IP), UDP: uint16(target.udpAddr().Port)})
    }

    report := lookupReport{}

    for {
        batch := selectBatch(shortlist, targetID, alpha)
        if len(batch) == 0 {
            break
        }
        step := lookupStep{}
        for _, idx := range batch {
            if shortlist[idx].queried {
                continue
            }
            shortlist[idx].queried = true
            remoteAgent := shortlist[idx].agent
            step.Queried = append(step.Queried, remoteAgent)

            nodes, err := source.sendFindnode(remoteAgent.udpAddr(), remoteAgent.id, target.pubkey)
            if err != nil {
                log.Printf("lookup: %s -> %s findnode error: %v", source.name, remoteAgent.name, err)
                continue
            }
            sort.Slice(nodes, func(i, j int) bool {
                return bytesCompare(nodes[i].ID, nodes[j].ID, targetID) < 0
            })
            step.Discovered = append(step.Discovered, nodes...)
            for _, node := range nodes {
                source.table.AddNode(node)
                addNode(node)
            }
            if remoteAgent.id == targetID {
                break
```

```go
            }
        }
        report.Steps = append(report.Steps, step)
    }

    return report, nil
}

// selectBatch는 아직 질의하지 않은 후보 중 타깃에 더 가까운 α개의 인덱스를 고른다.
func selectBatch(entries []lookupEntry, target enode.ID, alpha int) []int {
    type candidate struct {
        idx  int
        node kademlia.Node
    }
    cand := make([]candidate, 0)
    for i, entry := range entries {
        if entry.queried {
            continue
        }
        cand = append(cand, candidate{idx: i, node: entry.node})
    }
    sort.Slice(cand, func(i, j int) bool {
        return bytesCompare(cand[i].node.ID, cand[j].node.ID, target) < 0
    })
    if len(cand) > alpha {
        cand = cand[:alpha]
    }
    result := make([]int, len(cand))
    for i, c := range cand {
        result[i] = c.idx
    }
    return result
}

// bytesCompare는 두 노드의 XOR 거리를 비교해 정렬에 활용한다.
func bytesCompare(a, b enode.ID, target enode.ID) int {
    da := xorID(a, target)
    db := xorID(b, target)
    return bytes.Compare(da[:], db[:])
}

// xorID는 enode.ID 두 개의 XOR 결과를 반환해 Kademlia 거리 계산에 쓴다.
func xorID(a, b enode.ID) (out [32]byte) {
    for i := 0; i < len(out); i++ {
        out[i] = a[i] ^ b[i]
    }
    return out
}

// addrToNetip는 net.IP를 netip.Addr로 변환해 테이블에 저장할 수 있게 한다.
func addrToNetip(ip net.IP) netip.Addr {
    if ip == nil {
        return netip.Addr{}
    }
```

```go
    if v4 := ip.To4(); v4 != nil {
        return netip.AddrFrom4([4]byte{v4[0], v4[1], v4[2], v4[3]})
    }
    v6 := ip.To16()
    if v6 == nil {
        return netip.Addr{}
    }
    var arr [16]byte
    copy(arr[:], v6)
    return netip.AddrFrom16(arr)
}

```